COMP 3415 - Assignment 2

1. System Understanding

Purpose and scope
Campus Supply Desk is a local Python order-management application. It lets users view campus merchandise and stock, request quotes, place orders, review history, find orders, and cancel them. Regular and student customers can choose pickup or delivery; CR-01 adds express. Monetary values remain integer Canadian cents. Payments, taxes, addresses, authentication, and shipping workflows remain outside scope.

Components
cli.py defines build_parser(), parse_item(), and main() for individual commands and JSON output. menu.py collects requests through read_request() and displays products and orders. service.py contains OrderService for validation, pricing, stock, history, and cancellation. storage.py contains JsonStore.load() and save(); catalog.json supplies initial state. Tests use temporary save files.

Execution flow: cancellation
run_menu() dispatches cancellation to handle_cancel(). The user supplies an ID and confirms with y. cancel_order() loads state, finds the order, restores its item quantities, marks it cancelled, and saves through JsonStore. show_order() displays the result. Unknown IDs raise ValueError. The corrected service returns an already cancelled order without restoring stock again.

Maintainability assessment
Originally, run_menu() combined every operation, confirmation, and error path in one loop. The extracted handlers now isolate operations while the loop retains navigation and error handling. Quote and placement also repeated validation and pricing; both now use _prepare_request().

Repository: https://github.com/royalpriest/COMP3415-A2-Campus-Supply-Desk
Starter commit: 82ed75e0714e50c086a13ee491f51b349814d8a4.

2. Testing Strategy and Results

Approach

The suite uses Python unittest and a fresh temporary state path per test. Expected values come from FUNCTIONAL_SPEC.md and CR-01. Tests assert returned fields and state changes, rather than merely calling operations. Malformed save files are tested for an error without silent reset.

Normal and preservation cases

The starter student-delivery example checks an 8,000-cent subtotal, 800-cent discount, 600-cent fee, and 7,800-cent total. test_quote_matches_order_and_has_no_side_effects compares every quote field to placement and verifies that quotes do not create or modify saved state. History tests check reopening, numbering, cancellation status, original prices, and placement order.

Boundary and invalid cases

test_student_threshold covers below, exactly at, and above 5,000 cents. Delivery tests check the post-discount threshold; a controlled 5,005-cent price checks half-cent rounding. Tests order all available stock and use a controlled stock fixture for quantity 99. Invalid cases include booleans, strings, fractional quantities, unknown products, empty items, bad fulfillment, and partially valid requests; saved bytes and numbering must remain unchanged.

Observed stage results

Original suite: 4 tests passed. Expanded baseline: 22 tests, with four failed subtests across three defects. Structural refactoring: the same failures remained. Separate defect corrections: all 22 passed. CR-01 tests before implementation failed because express was unsupported. Final suite: all 28 passed, without skips or expectedFailure markers.

Evidence

evidence/01-original-tests.txt through 06-after-cr01.txt retain actual stage output. Duplicate stock validation fails in both quote and placement, accounting for two of the four baseline failures. evidence/07-process-verification.txt checks separate-process CLI quote, placement, lookup, repeated cancellation, rejected input, and interactive express quoting. Run: python -m unittest discover -s tests -v .

3. Five Code Problem Diagnoses

1. Duplicated code
Location: original service.py, quote() and place_order(). Evidence: repeated customer/fulfillment validation, product lookup, quantity checks, line construction, discounts, and delivery calculation. Consequence: updates can leave quotes and placements inconsistent. Improvement: extract shared preparation using the caller's loaded state, leaving mutation and saving in placement.

2. Long method
Location: original menu.py, run_menu(). Evidence: one function implements all menu choices and their prompts, confirmation, service calls, and rendering. Consequence: adding an operation makes navigation and maintenance harder. Improvement: extract operation handlers and retain central navigation, exits, and error handling.

3. Magic numbers
Location: original service.py pricing and validation expressions. Evidence: 99, 5000, 10, 50, 100, 10000, and 600 are embedded in rules and calculations. Consequence: their roles require interpretation and duplicated policies can change inconsistently. Improvement: named constants for quantity, monetary thresholds, percentages, rounding offset, and fees, retaining integer arithmetic.

4. Storage coupling / testability
Location: OrderService.__init__(). Evidence: it directly constructs JsonStore(state_path). Consequence: in-memory test stores and simulated save failures need patching or replacement after construction. Proposed improvement: optionally accept a store dependency. This diagnosis remains documented; dependency injection was not necessary for the selected three refactorings or CR-01. Tests instead use real temporary files.

5. Combined-quantity defect
Location: original quote() and place_order() item loops. Evidence: each notebook line of 15 passes against stock of 20; two lines together request 30. Placement then deducts both and can save -10 stock. Consequence: unavailable requests are approved and inventory becomes invalid. Improvement: accumulate quantities by product and reject overstock before mutation. This is a behavioral correction, not refactoring. Additional discovered defects are recorded separately in Task 4.

4. Refactoring and Verification

Refactoring 1: shared request preparation
Before: quote() and place_order() contained parallel validation and pricing implementations. Change: _prepare_request(state, customer_type, items, fulfillment) returns item lines and pricing; both operations use it. Placement alone constructs ID/status, deducts stock, increments numbering, appends history, and saves. Result: business rules have one implementation while quotes remain read-only. Verified by agreement, side-effect, invalid-request, numbering, and persistence tests.

Refactoring 2: menu operation handlers
Before: run_menu() mixed the implementation of all operations. Change: extracted handle_catalog(), handle_quote(), handle_place(), handle_history(), handle_lookup(), and handle_cancel(); the loop dispatches by choice. Result: individual flows can be read independently while exit and recovery behavior remain centralized. Verified by adapter tests for all options, confirmation declines, invalid input recovery, defaults, EOF, and KeyboardInterrupt.

Refactoring 3: named constants
Before: pricing and quantity limits used unexplained literals. Change: named constants explain policy and integer percentage rounding. Result: thresholds and fees are easier to identify. The structural stage deliberately retained the original strict > discount comparison. Threshold, fee, and rounding tests check behavior.

Separately corrected defects
After structural refactoring, changed > to >= at the existing $50 student threshold; accumulated requested quantities per product; and returned early on already cancelled orders. These intentionally change defective behavior to meet the existing specification. They are not claimed as behavior-preserving refactorings and do not represent new CR-01 policy.

Evidence and limits
The structural snapshot is evidence/03-refactoring.patch; do not apply it on top of final code. Commit 202cce3 retains the same four failed subtests as baseline 3440201. Commit 206ebcf fixes the defects and passes 22 tests. This is evidence for tested behavior, not a proof of every possible input. Single-writer and save-file recovery limits remain as specified.

5. Client Change Request: Express Delivery

Prediction before implementation
docs/CR-01-prediction.md records the expected changes: service validation and delivery calculation; CLI choices/help; menu prompt; README and functional specification; and acceptance tests. Storage format and lookup/cancellation were predicted to need no express-specific change because orders already store fulfillment and price snapshots.

Implementation
service.py accepts express and charges EXPRESS_DELIVERY_FEE_CENTS = 1200 in shared preparation. The fee is never waived. Student discounts apply only to merchandise, using the existing inclusive eligibility threshold and integer half-up rounding. cli.py exposes express in choices and help; menu.py includes the new choice and price in its prompt. Pickup remains the default and standard delivery rules remain intact.

Acceptance evidence
test_cr_examples_and_thresholds verifies totals of 2200 cents for two regular notebooks, 8400 for two student hoodies, and 13200 for three regular hoodies; standard delivery for three hoodies remains 12000. Further subcases verify the fixed fee at the free-standard-delivery threshold and the exact student discount boundary. test_express_half_cent_rounding_and_fee_not_discounted verifies a 501-cent discount on 5005 cents of merchandise plus the full express fee.

Lifecycle and interfaces
test_express_lifecycle_preserves_snapshots_and_numbering checks quote/order agreement, reopening, history, stock, cancellation snapshots, repeated cancellation, and the next ID. Other tests exercise CLI quote/place/help, menu quote/place and prompt discoverability, unsupported fulfillment, invalid quantities, duplicate stock, and failure atomicity. Separate-process checks confirm the actual module entry point.

Prediction review
The actual source changes matched the prediction: service.py, cli.py, menu.py, README.md, FUNCTIONAL_SPEC.md, and tests/test_express.py. storage.py and catalog.json were unchanged. Shared preparation made the change easier because express validation and pricing were added once. Commit fcadf6e records the prediction and failing acceptance cases; 39058aa implements express with all 28 tests passing.

6. AI Engineering Judgment and Submission

Decisions and interaction record
My decisions are ACCEPT, MODIFY, and ACCEPT. These cover two decision categories. The recommendation excerpts and implemented actions are recorded in docs/AI-RECORD.md.

Recommendation 1 - ACCEPT
Recommendation: extract shared request-preparation and pricing helpers. I accepted this because quote() and place_order() apply the same validation and pricing rules. Keeping two implementations risks inconsistent results. Both now use _prepare_request(), while placement alone changes stock and saves. The 22-test baseline and structural-stage run retained the same known failures; quote/placement agreement and state-preservation tests protected the shared behavior.

Recommendation 2 - MODIFY
Recommendation: allow an optional supplied store while retaining JsonStore by default. I modified this approach by keeping JSON storage and testing with temporary save files. Dependency injection could help future tests, but it was unnecessary for the selected refactorings and CR-01. Tests already verify reopening, saved history, and byte-for-byte nonmutation after invalid requests. The storage-coupling diagnosis remains documented without adding an unused injection interface.

Recommendation 3 - ACCEPT
Recommendation: track existing defects separately from refactoring. I accepted this because correcting behavior differs from preserving it. FUNCTIONAL_SPEC.md requires the inclusive $50 discount threshold, combined-stock validation, and one-time stock restoration. The baseline and structural runs exposed four failed subtests across these three defects. A separate correction stage passed all 22 tests before express delivery was added.

Repository and development history
Public fork: https://github.com/royalpriest/COMP3415-A2-Campus-Supply-Desk . Changes were initially prepared uncommitted, then reconstructed and rechecked in six ordered implementation commits. evidence/08-commit-verification.md records this process. Later documentation commits refine the report and record these confirmed decisions. The last implementation commit is 39058aa; subsequent changes affect documentation only.

Verification and submission
The final implementation passes all 28 tests using python -m unittest discover -s tests -v . Separate-process checks are in evidence/07-process-verification.txt. Submit this PDF, the public repository link, testing evidence, and AI-use record through MyCourseLink. No PR to the instructor repository is required.